Data and Algorithm Analysis

Chapter 21 — Minimum Spanning Trees

Thang Hoang

Department of Comptuer Science, Virginia Tech

Outline

MSTs

Generic Greedy Algorithms

Kruskal's Algorithm

Prim's Algorithm

Table of Contents

MSTs

Generic Greedy Algorithms

Kruskal's Algorithm

Prim's Algorithm

Definitions — MST

Let $G = (V, E)$ be an undirected, connected graph. A **spanning tree** of G is a set $T \subseteq E$ such that the subgraph (V, T) is acyclic and connected.

Question: If $|V| = n$, then what is $|T|$?

Let $w : E \to \mathbb{R}$ be a weight function on the edges of G. Then, the weight of T is

$$w(T) = \sum_{(u,v) \in T} w(u, v).$$

A **minimum spanning tree (MST)** of G is a spanning tree T of minimum weight.

Problem — MST

MINIMUM SPANNING TREE (MST)

Input: Undirected, connected graph $G = (V, E)$; weight function $w : E \to \mathbb{R}$.

Output: Spanning tree $T \subseteq E$ of G such that

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

is minimum.

Example for MST

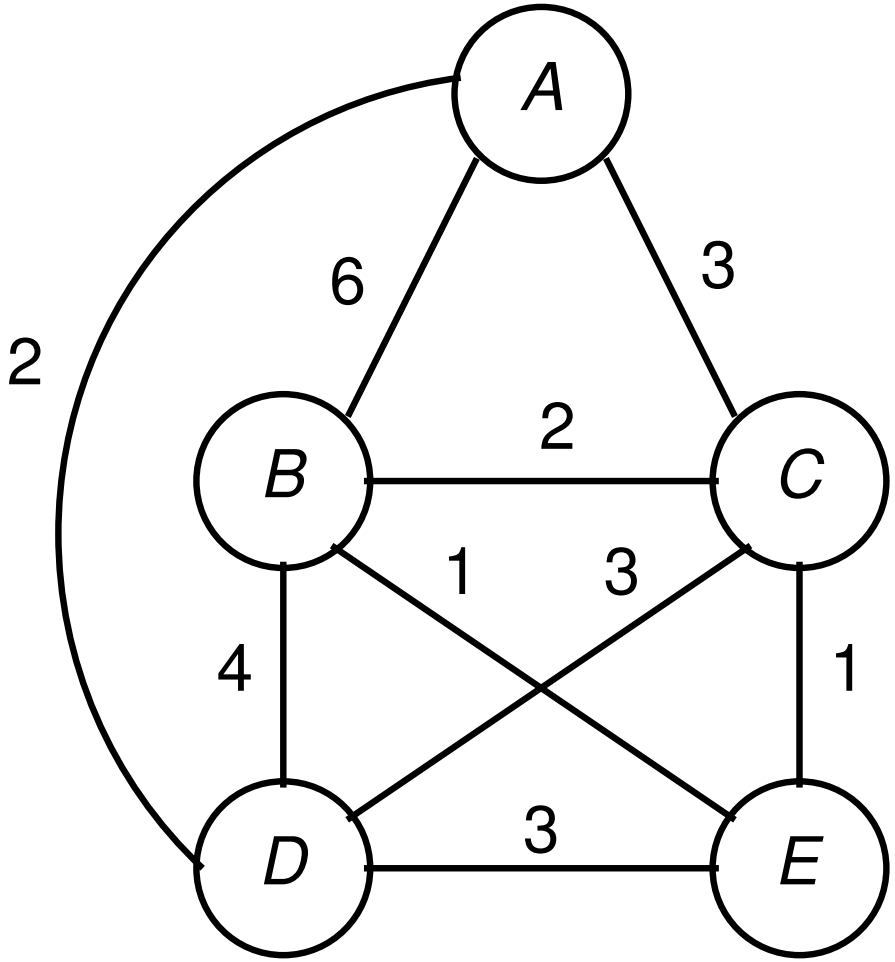


Table of Contents

MSTs

Generic Greedy Algorithms

Kruskal's Algorithm

Prim's Algorithm

Definitions — Safe Edges

Let $A \subseteq E$ be a set of edges that is contained in some MST of $G = (V, E)$. An edge $(u, v) \in E \setminus A$ is a **safe edge** for A if $A \cup \{(u, v)\}$ is contained in some MST.

Pseudocode — Generic Greedy MST

```
GENERIC-MST(G, w)
    A = ∅
    while A does not form a spanning tree
        select (u, v) ∈ E \ A that is safe for A
        A = A ∪ {(u, v)}
    return A
```

Definitions — Light Edges

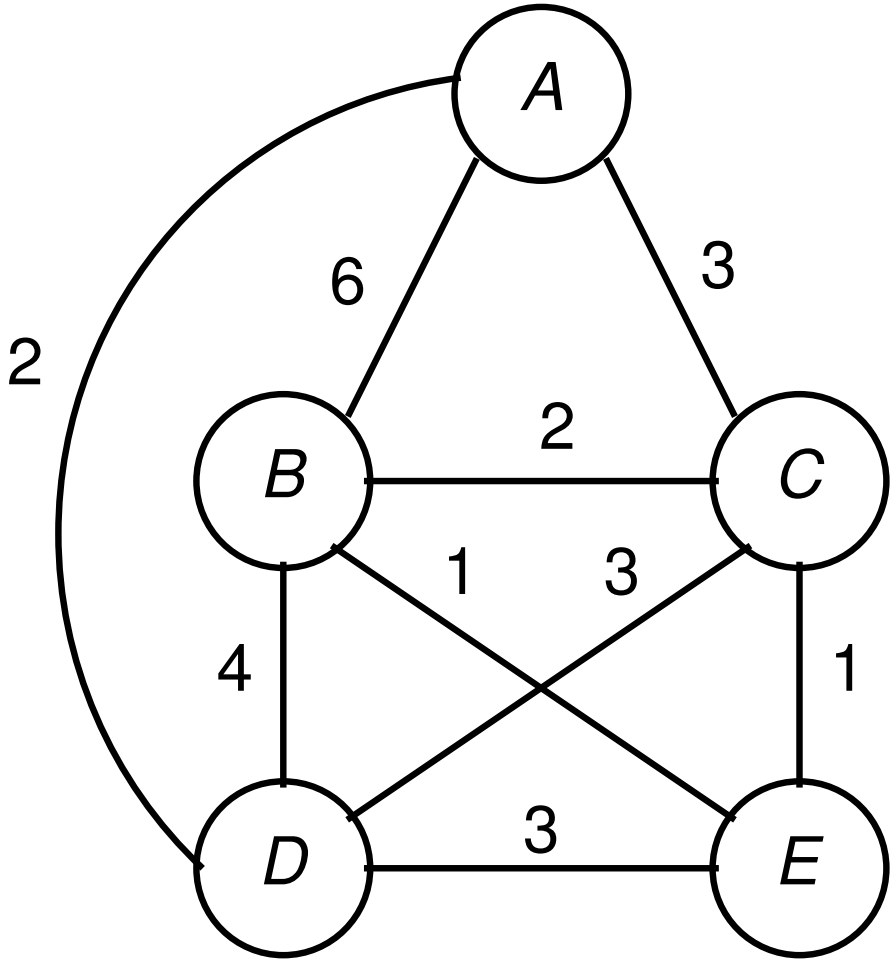
If $S \subseteq V$, $S \neq \emptyset$, and $S \neq V$, then the partition $(S, V \setminus S)$ of V is a **cut** of G.

An edge (u, v) **crosses the cut** $(S, V \setminus S)$ if $u \in S$ and $v \in V \setminus S$ or vice versa.

The cut $(S, V \setminus S)$ **respects** $A \subseteq E$ if no edge in A crosses the cut.

A **light edge** for a cut $(S, V \setminus S)$ is an edge crossing the cut of minimum weight.

Example for Cuts and Light Edges



Theorem 21.1

Theorem

Let $G = (V, E)$ be an undirected, connected graph with edge weight function $w : E \to \mathbb{R}$. Let $A \subseteq E$ be contained in some MST of G. Let $(S, V \setminus S)$ be any cut respecting A. Let (u, v) be a light edge crossing $(S, V \setminus S)$. Then, (u, v) is safe for A.

Proof of Theorem 21.1

Proof.
Let $T \subseteq E$ be an MST containing A. If $(u, v) \in T$, then (u, v) is safe for A, as desired. Otherwise, $(u, v) \notin T$. Then, construct another spanning tree T' of G as follows.
(See continuation of the proof on the board.) $\square$

Corollary 21.2

Corollary

Let $G = (V, E)$ be an undirected, connected graph with edge weight function $w : E \to \mathbb{R}$. Let $A \subseteq E$ be contained in some MST of G but not a spanning tree. Then, $F = (V, A)$ is a forest. Let $C = (V_C, E_C)$ be a connected component of F. If (u, v) is such that $u \in V_C$, $v \in V \setminus V_C$, and $w(u, v)$ is minimum, then (u, v) is safe for A.

Proof.

Just take $S = V_C$ in Theorem 21.1. □

Pseudocode — Light Edge MST

```
LIGHT-EDGE-MST(G, w)
    preprocessing to initialize data structure
    A = ∅
    while A does not form a spanning tree
        select a cut (S, V \ S) of G that respects A
        select (u, v) a light edge crossing (S, V \ S)
        A = A ∪ {(u, v)}
        update data structure
    return A
```

Table of Contents

MSTs

Generic Greedy Algorithms

Kruskal's Algorithm

Prim's Algorithm

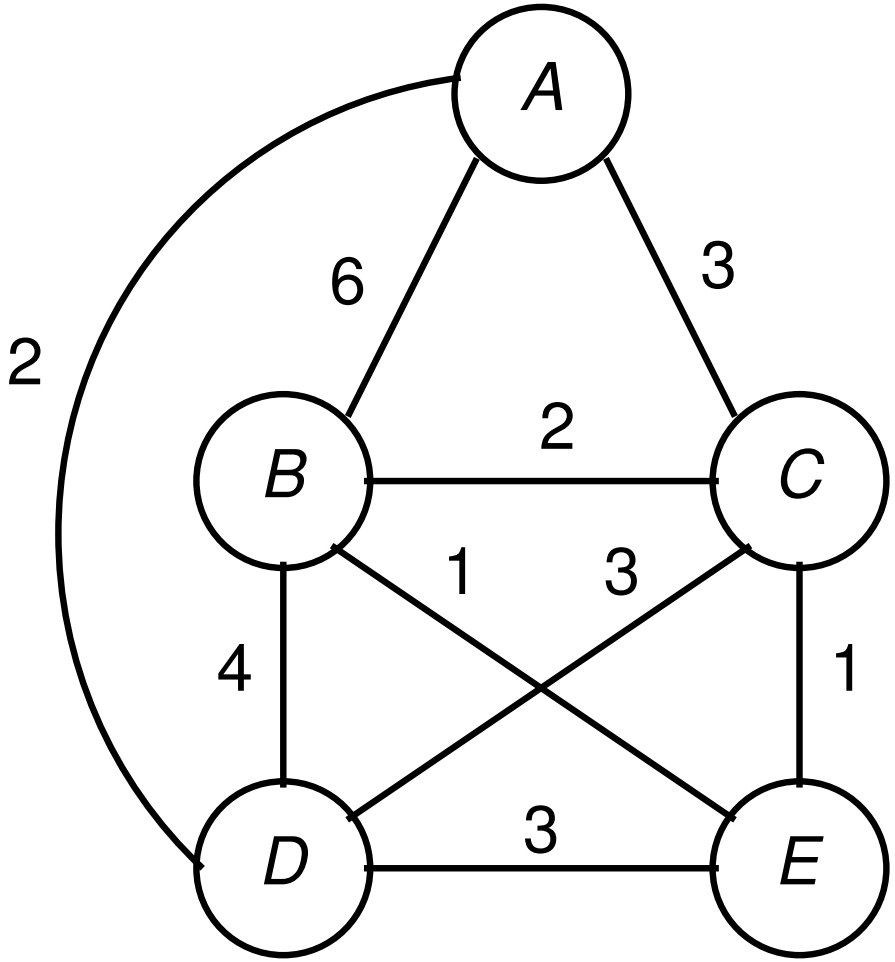
Intuition — Kruskal's Algorithm

- Process edges in nondecreasing order by weight and select an edge if it does not create a cycle.
- Maintain the connected components of the spanning forest to identify whether or not a cycle will be created.
- Use a data structure to efficiently maintain the connected components as a partition of V under these operations:
  - MAKE-SET(v)
  - UNION(u, v)
  - FIND(u)

Pseudocode — Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
    A = ∅
    for v ∈ V
        MAKE-SET(v)
    sort E in nondecreasing order by weight w
    for (u, v) ∈ E in nondecreasing order by weight
        if FIND-SET(u) ≠ FIND-SET(v)
            A = A ∪ {(u, v)}
            UNION(u, v)
    return A
```

Example for Kruskal's Algorithm



Disjoint Sets Data Structure

Implementation: Disjoint set forest to represent a partition $\{V_1, V_2, \ldots, V_k\}$ of V. The vertices of each V_i are the vertices of a rooted tree with edges pointing toward the root and the **canonical element** at the root; each vertex v has a parent pointer $v.parent$.

- MAKE-SET(v): initially make each V_i a singleton tree, one for each $v \in V$.
- UNION(u, v): join two disjoint trees into a new tree.
- FIND(u): follow parent pointers to find the canonical element for the set.

Example on the board.

Time complexity of operations: ?

Time Complexity

Time complexity of Kruskal's Algorithm: ?

Table of Contents

MSTs

Generic Greedy Algorithms

Kruskal's Algorithm

Prim's Algorithm

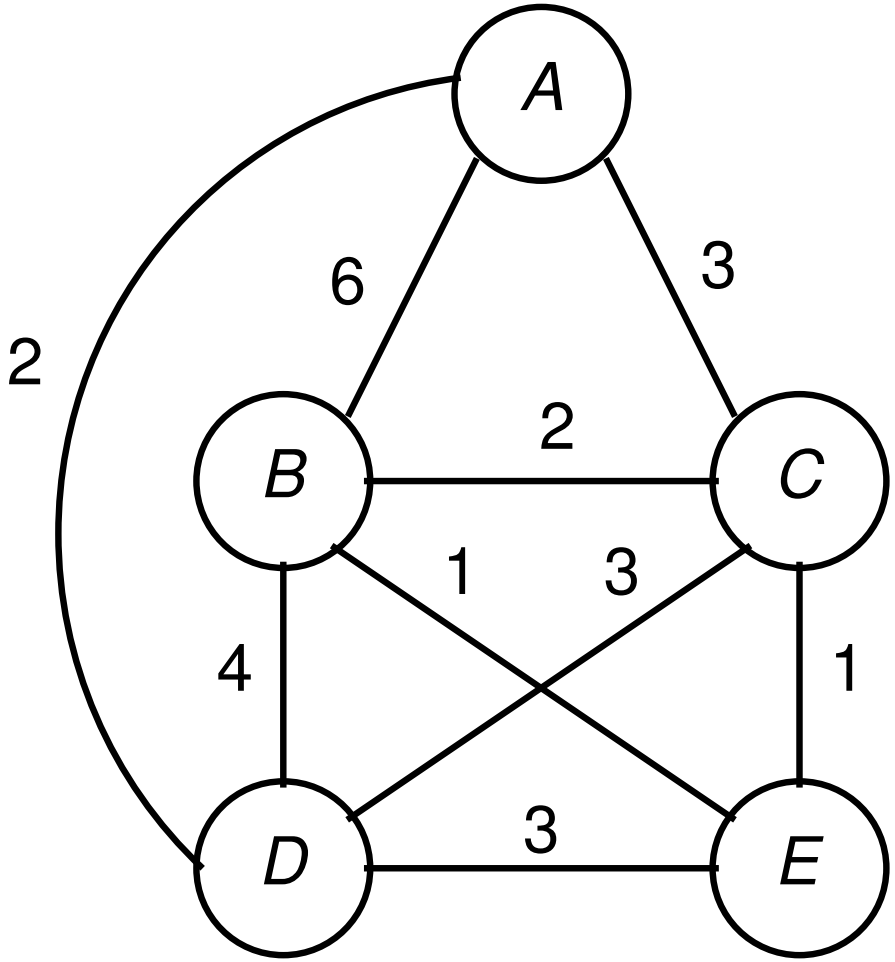
Intuition — Prim's Algorithm

- Grow a single tree in the spanning forest one edge at a time.
- The cut is from that the vertices spanned by that one tree to the rest of V.
- Use a **min priority** queue to choose a light edge crossing the cut.

Pseudocode — Prim's Algorithm

```
MST-PRIM(G, w, r)
for u ∈ V
    u.key = ∞
    u.π = NIL
r.key = 0
Q = V
while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        for v adjacent to u
            if v ∈ Q and w(u, v) < v.key
                v.π = u
                v.key = w(u, v)
return u.π for u ∈ V
```

Example for Prim's Algorithm



Time Complexity

Time complexity of Prim's Algorithm: ?